

07 — Running guide (every section, one by one)

How to actually run each part of ProvenanceRank — the ranking engine, the tests, the accuracy + performance layers, the API, the frontend, the live GitHub layer, the graph RAG search, the full Docker stack, and observability. Copy-paste commands throughout.

There are three “levels” and you can stop at whichever you need:

- **Level 1 — the engine only** (Sections 1-5): produces the graded `submission.csv`. Needs only Python.
- **Level 2 — the app, no Docker** (Sections 6-9): the API + UI on SQLite + in-memory fallbacks. Needs Python + Node.
- **Level 3 — the full stack** (Section 10): Postgres, Redis, Neo4j, workers, nginx, Prometheus, Grafana, Jaeger, the UI. Needs Docker.

Shortcut: a Makefile wraps the common commands. Run `make help` to list them. Each section below notes the make target where one exists.

Quick start — Docker Desktop (the fewest steps)

Recommended on a laptop: the lite stack, from a clean state

The lite stack is 6 containers instead of 14 — far less disk/RAM and far fewer ways to fail. If a previous run got into a bad state, start clean:

```
cd /Users/herender/Desktop/iCode/LinkedIn_finder/provenancerank
```

```
# 1. dataset must be a real FILE at the repo root (one-time)
```

```
[ -f candidates.jsonl ] || unzip -p "../[PUB] India_runs_data_and_ai_challenge.zip" \
  "*/candidates.jsonl" > candidates.jsonl
```

```
# 2. reclaim Docker disk + clear any half-built volumes from earlier attempts
```

```
docker system prune -af
```

```
docker compose -f docker-compose.lite.yml down -v
```

```
# 3. (optional) live-layer tokens – set BEFORE 'up'
```

```
export GITHUB_TOKEN=ghp_XXX
```

```
export GOOGLE_API_KEY=AIza_XXX
```

```
# 4. bring it up (first run builds images + the feature matrix; ~a few minutes)
```

```
docker compose -f docker-compose.lite.yml up --build
```

```
# 5. wait until the ranker is ready, THEN open the UI:
```

```
curl -s localhost:8080/health/ready # ok = ready
```

```
# open http://localhost:3001 (log in: admin@provenancerank.local / changeme-admin)
```

Stop it with `docker compose -f docker-compose.lite.yml down`.

Don't click **Run ranking** until `feature_matrix.pkl` exists and `/health/ready` is ok — otherwise it ranks 0 candidates and fails. Check: `docker compose -f docker-compose.lite.yml exec api1 ls -lh /app/artifacts | grep feature_matrix`

Full stack (all 14 services)

If Docker Desktop is running and you want the complete stack (Neo4j, workers, Prometheus/Grafana/Jaeger), run these **in order**, once:

```
# 1. go to the repo
```

```
cd /Users/herender/Desktop/iCode/LinkedIn_finder/provenancerank
```

```
# 2. make sure the dataset is a real FILE at the repo root (one-time).
# If you see a "candidates.jsonl" *directory*, remove it first: rm -rf candidates.jsonl
[ -f candidates.jsonl ] || unzip -p "../[PUB] India_runs_data_and_ai_challenge.zip" \
  "*/candidates.jsonl" > candidates.jsonl

# 3. (optional) tokens for the live layer – set BEFORE 'up'
export GITHUB_TOKEN=ghp_xxx          # GitHub ingestion
export GOOGLE_API_KEY=AiZa_xxx       # Gemini JD parse + summaries

# 4. bring up the entire stack (first run builds images; later runs are fast)
docker compose up --build             # or: make up
```

Then just open:

URL	What
http://localhost:3001	recruiter UI
http://localhost:8080/docs	API (via the nginx gateway)
http://localhost:7474	Neo4j browser (neo4j / provenancerank)
http://localhost:3000	Grafana · http://localhost:16686 Jaeger · :9090 Prometheus

You do **not** run precompute, uvicorn, or npm yourself — the stack runs the precompute container automatically and starts all 13 services. Stop with `docker compose down` (add `-v` to wipe data). Full details + the lean-image caveat are in **Section 10**.

First boot builds images and runs precompute, so give it a few minutes. The precompute step uses `--resume`, so on later ups it reuses the cached artifacts (stored in a Docker volume) and skips the rebuild.

0. One-time setup

```
cd /Users/herender/Desktop/iCode/LinkedIn_finder/provenancerank
```

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt          # or: make install
```

Put the dataset at the repo root (it's large, so it's gitignored):

```
cp /path/to/candidates.jsonl ./candidates.jsonl
ls -la candidates.jsonl          # must be a FILE (not a directory), ~480 MB
```

Every heavy dependency is optional — without them the code uses a pure-Python fallback (see [02_architecture.md](#), the degradation ladder). For a fast, lean install that skips torch/Gemini:

```
grep -viE "sentence-transformers|google-generativeai" requirements.txt > req.lean.txt
pip install -r req.lean.txt
```

1. The ranking engine → submission.csv (the graded core)

Two phases. **Phase 1** (offline, slow OK) builds artifacts; **Phase 2** (the graded step: no network, CPU, < 5 min) reads them.

Phase 1 – build features, embeddings, BM25, the model, rerank + tournament

```
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt
```

```
# make precompute

# Phase 2 – the graded step
python rank.py --candidates ./candidates.jsonl --out ./submission.csv
# make rank

# verify against the official validator
python validate_submission.py submission.csv # -> "Submission is valid."
```

Useful precompute.py flags:

Flag	What it does
--no-st	skip sentence-transformers, use the hashing embedder (≈30s on 100K)
--bm25-backend inverted	memory-frugal keyword index (pairs well with --no-st)
--no-accuracy	skip the rerank/tournament layer (original proxy regressor)
--resume	reuse cached artifacts whose inputs are unchanged
--limit N	only process the first N candidates (dev only)

Fast offline run: `python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt --no-st --bm25-backend inverted`.

Do I need to re-run precompute every time?

No. Precompute is a one-time build per (*dataset, JD*). It writes everything to artifacts/; after that, run `rank.py` as many times as you like — it just reads the cache and finishes in ~2 seconds. You only re-run precompute when an **input changes**:

What changed	Re-run precompute?	Cost (with --resume)
Nothing	No — just run <code>rank.py</code>	—
The job description only	Yes, with --resume	seconds — reuses the 100K embeddings + the trained model, only redoes retrieval/rerank/tournament
The candidate dataset	Yes, with --resume	rebuilds (inputs changed)
Engine code / config	Yes	rebuilds the affected parts

Always add --resume after the first build — it hashes the inputs and recomputes only what actually changed (and skips entirely, ~1s, if nothing did). So your slow 8-minute embedding step is paid **once**:

```
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt --resume
python rank.py --candidates ./candidates.jsonl --out ./submission.csv # repeat freely
```

2. Tests & lint

```
python -m pytest -q # the whole suite (68 tests) | make test
python -m pytest tests/test_accuracy_layer.py -q # one file
python -m pytest -k tournament -q # by keyword

python -m ruff check . # lint | make lint
python -m ruff check . --fix # auto-fix
```

3. The accuracy layer (measure the lift)

The layer (graded-label LambdaMART + cross-encoder rerank + Bradley-Terry tournament) runs automatically inside `precompute.py`. To **prove** it helps, A/B it with the eval harness (`ml/evaluate.py`):

```
# baseline (layer off)
```

```
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt --no-acc
```

```
python rank.py --candidates ./candidates.jsonl --out ./submission_base.csv
```

```
# full (layer on)
```

```
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt
```

```
python rank.py --candidates ./candidates.jsonl --out ./submission_full.csv
```

To compute NDCG@10/@50, MAP, P@10 yourself, import the harness:

```
from ml.evaluate import evaluate
```

```
metrics = evaluate(ranked_ids, relevance) # relevance = {candidate_id: grade}
```

```
print(metrics["composite"])
```

Full detail: [../ACCURACY.md](#).

4. Performance levers (speed, no feature loss)

All controlled by environment variables (or `core/config.py`); see [../ACCURACY.md](#) “Keeping it fast”.

```
export COMPUTE_DEVICE=auto # auto | cpu | cuda | mps (GPU auto-detected)
```

```
export USE_ONNX=true # ONNX Runtime + int8 (needs optimum[onnxruntime])
```

```
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt --resume
```

--resume + the content-hash cache means a second run with unchanged inputs is near-instant, and changing only the JD reuses the 100K embeddings (look for `cache.hit` in the logs).

5. (Optional) build a small sample dataset

For quick iteration without the full 100K:

```
python scripts/make_dev_sample.py # writes a stratified data/dev_sample.jsonl
```

```
python precompute.py --candidates ./data/dev_sample.jsonl --jd ./data/job_description.txt --n
```

6. The API (no Docker)

The API falls back to **SQLite + in-memory cache/graph/queue**, so nothing else needs to be installed.

```
source .venv/bin/activate
```

```
uvicorn api.main:app --reload --port 8000 # make serve
```

```
# open http://localhost:8000/docs (interactive Swagger UI for every endpoint)
```

A **bootstrap admin** is created on first boot:

- email: `admin@provenancerank.local`
- password: `changeme-admin`

Auth flow (curl)

```
# 1) log in -> get a JWT
TOKEN=$(curl -s -X POST localhost:8000/auth/login \
  -H 'Content-Type: application/json' \
  -d '{"email":"admin@provenancerank.local","password":"changeme-admin"}' \
  | python3 -c "import sys,json;print(json.load(sys.stdin)['access_token'])")

# 2) call an authed endpoint
curl -s localhost:8000/auth/me -H "Authorization: Bearer $TOKEN"

# 3) (optional) mint an API key for scripts/machines
curl -s -X POST localhost:8000/auth/api-keys -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"name":"my-script"}'
# then send it as: -H "X-API-Key: <the key shown once>"
```

Endpoint map

Method & path	What
GET /health, /health/live, /health/ready	liveness / readiness
POST /auth/login, /auth/register, GET /auth/me	auth
POST /auth/api-keys, GET /auth/api-keys	API keys
POST /rank	submit a ranking job → job_id
GET /rank, GET /rank/{job_id}	list / status
GET /rank/{job_id}/results	the ranked top-100
GET /rank/{job_id}/submission.csv	download the CSV
GET /rank/{job_id}/stream	live progress (SSE)
POST /developers/connect-github	live layer — ingest a developer
GET /developers/{id}/sync-status	real sync status + counts
GET /developers/{id}/evidence-graph	the proof-of-work subgraph
POST /search/natural-language	graph RAG search (English)
GET /search/by-skill/{skill}	structured skill lookup
GET /admin/artifacts, POST /admin/precompute, GET /admin/jobs, /admin/audit	admin
GET /metrics	Prometheus metrics

Run a ranking via the API

```
curl -s -X POST localhost:8000/rank -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"top_k":100}'
# -> {"job_id":"..."} ; then poll:
curl -s localhost:8000/rank/<job_id> -H "Authorization: Bearer $TOKEN"

(Needs the Phase-1 artifacts from Section 1 to exist.)
```

7. The frontend (recruiter UI, no Docker)

```
cd frontend
npm install
npm run dev
# open http://localhost:5173
```

Log in with the bootstrap admin above. Pages: **Dashboard** (run a ranking), **Rankings** (results), **Evidence Search** (the live/graph layer), **Settings** (API keys). The dev server proxies API calls to localhost:8000, so run the API (Section 6) alongside it.

8. The live GitHub layer (evidence ingestion)

This is the differentiator; the core ranking does **not** need it. To ingest real activity you must give the **server process** a GitHub token:

```
export GITHUB_TOKEN=ghp_your_personal_access_token # classic PAT; read-only OK
export GOOGLE_API_KEY=AIza_xxx # optional: Gemini summaries
uvicorn api.main:app --reload --port 8000 # restart so it sees the token
# verify the server actually sees it:
python -c "from core.config import get_settings; print('token seen:', bool(get_settings().git
```

Then ingest and check the **real** status (don't trust the UI's "Syncing..." label):

```
# connect a developer
curl -s -X POST localhost:8000/developers/connect-github -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' -d '{"github_username":"octocat"}'
# -> {"developer_id":"...", "status":"syncing"}

# the truth: synced / error + how many artifacts were ingested
curl -s localhost:8000/developers/<developer_id>/sync-status -H "Authorization: Bearer $TOKEN"

# the proof-of-work subgraph
curl -s localhost:8000/developers/<developer_id>/evidence-graph -H "Authorization: Bearer $TOKEN"
```

Without a token, ingestion fails fast and the developer goes to error (the server log shows `developer.sync_failed`). Background work runs **eager** (inline) unless you run the Celery workers (Section 10).

9. Graph RAG / natural-language search

Ask in English; the system fuses exact skill traversal with semantic vector search over evidence summaries (see [03_features.md](#) E4).

```
curl -s -X POST localhost:8000/search/natural-language -H "Authorization: Bearer $TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"query":"who built a recommendation system at scale"}'
```

Each result is tagged `match_via` (skill / semantic / both). Flags (env vars): `GRAPH_HYBRID_ENABLED` (default true), `GRAPH_VECTOR_TOP_K`, `GRAPH_RRF_K`, `NEO4J_ENABLED` (false = in-memory graph, still fully functional).

Measure precision/recall with the graph eval harness:

```
python -m pytest tests/test_graph_rag.py -q # baseline vs hybrid, asserts the lift
# or import graph.evaluate (precision@k, recall@k, MRR, compare, lift)
```

10. The full production stack (Docker)

Needs **Docker Desktop**. Brings up 13 services.

```
cd /Users/herender/Desktop/iCode/LinkedIn_finder/provenancerank
# candidates.jsonl must be a real FILE at the repo root (see Section 0) –
# if it's missing, Docker creates an empty *directory* and precompute fails.
docker compose up --build # make up / make down to stop
```

Set tokens for the workers (in your shell, an `.env`, or the compose env):

```
export GITHUB_TOKEN=ghp_xxx
export GOOGLE_API_KEY=AIza_xxx
```

Where everything lives

URL	Service
http://localhost:3001	recruiter UI (frontend)
http://localhost:8080/docs	API via the nginx gateway
http://localhost:7474	Neo4j browser (neo4j / provenancerank)
http://localhost:9090	Prometheus
http://localhost:3000	Grafana (anonymous)
http://localhost:16686	Jaeger traces

Boot order is handled for you: precompute runs once to build artifacts, then the API replicas, workers, and beat start. `docker compose down -v` also wipes the data volumes.

10b. Lite stack (6 containers instead of 14)

On a smaller machine the full 14-container stack is heavy. `docker-compose.lite.yml` runs the **same app** with far fewer moving parts — great for demos and laptops:

```
docker compose -f docker-compose.lite.yml up --build
# open http://localhost:3001 (UI) and http://localhost:8080/docs (API)
docker compose -f docker-compose.lite.yml down          # stop it
```

It keeps Postgres, Redis, precompute, one API, the nginx gateway, and the UI. It drops **neo4j** (graph falls back to the in-memory store), the **Celery workers + beat** (ingestion runs inside the API; only the auto-scheduled re-sync is gone), and **Prometheus/Grafana/Jaeger** (the API still emits metrics/logs/traces — you just don't run the dashboards). **No product feature is lost** and the graded submission.csv is unaffected. Switch back to the full stack any time with the normal `docker compose up`.

11. Observability

What	How to run / view
Metrics	GET /metrics on the API; scraped by Prometheus (:9090)
Dashboards	Grafana at :3000 (provisioned from <code>deploy/grafana/</code>)
Tracing	enable <code>OTEL_ENABLED=true</code> ; view spans in Jaeger (:16686)
Logs	structured JSON to stdout; make <code>logs</code> tails the API container

12. Database & migrations

Dev uses SQLite automatically. For Postgres (prod), apply migrations:

```
alembic upgrade head          # make migrate
export DATABASE_URL="postgresql+asyncpg://user:pass@localhost:5432/provenancerank"
make clean removes caches + the local SQLite DB to start fresh.
```

LLM provider — GLM first, local Ollama fallback

All four LLM touchpoints — JD parsing, evidence summaries, pseudo-label grading, and natural-language query parsing — go through one helper (`core/llm.py`). In the default auto mode it tries, in order: **GLM** (your Z.ai/Zhipu key) → **local Ollama** → **Gemini** → the deterministic heuristic. So your GLM key is used first, and if a call fails or the key is unset it falls back to a local model.

```
# primary: GLM (Z.ai / Zhipu)
export GLM_API_KEY=your_glm_key
export GLM_MODEL=glm-5.2 # default

# fallback: local Ollama (used if GLM is unset or a call fails)
brew install ollama && ollama serve
ollama pull llama3.1:8b # or qwen3-coder:30b on a 24GB+ Mac

python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt --resume

Force a single provider with LLM_PROVIDER=glm (or ollama / gemini / none). GLM is called over its
OpenAI-compatible endpoint, so GLM_BASE_URL can point at any compatible API.
```

Use the LLM to improve the ranking (LLM_HEAD_SCORING_ENABLED)

Beyond JD parsing, the LLM can directly score the **top of the candidate list**. Enabled, precompute has the LLM read the top ~250 candidates against the JD, score fit 0-100, cache it as `llm_fit_score`, and the scorer blends it into the head (weight `LLM_SCORE_WEIGHT`, default 0.15). `rank.py` only reads the cached column, so it stays no-network/<5min.

```
export GLM_API_KEY=your_glm_key
export LLM_HEAD_SCORING_ENABLED=true # off by default
python precompute.py --candidates ./candidates.jsonl --jd ./data/job_description.txt
```

```
# one-command A/B: builds both rankings from the same matrix, prints the metric
# delta, and writes submission_base.csv + submission_llm.csv to eyeball the names
python scripts/ab_llm.py
```

`ab_llm.py` scores with-vs-without the LLM column against the graded pseudo-labels and prints composite / ndcg@10 lift + head churn — so you only keep it if it actually helps. It costs ~a dozen LLM calls offline; with no LLM reachable the whole thing is a silent no-op.

In Docker, Ollama runs on the host, so point the containers at it:

```
export OLLAMA_BASE_URL=http://host.docker.internal:11434
export LLM_PROVIDER=ollama OLLAMA_MODEL=llama3.1:8b
docker compose -f docker-compose.lite.yml up --build
```

The LLM steps are still optional. `precompute/rank` produce a valid `submission.csv` with no LLM at all (rule-based JD parse + heuristic labels).

Environment-variable quick reference

Variable	Default	Controls
GITHUB_TOKEN	—	live-layer GitHub ingestion (required for it)
LLM_PROVIDER	auto	which LLM to use: auto (GLM → Ollama → Gemini) / glm / ollama / gemini / none
GLM_API_KEY	—	Z.ai / Zhipu GLM key — used first in auto when set
GLM_MODEL	glm-5.2	GLM model name
GLM_BASE_URL	https://api.z.ai/api/paas/v4	any OpenAI-compatible endpoint
OLLAMA_BASE_URL	http://localhost:11434	local Ollama fallback (Docker: http://host.docker.internal:11434)
OLLAMA_MODEL	llama3.1:8b	any model you've ollama pull-ed
GOOGLE_API_KEY	—	Gemini (last resort if neither GLM nor Ollama is available)
PSEUDO_LABEL_LLM_ENABLED	false	opt in to LLM <i>label grading</i> in precompute (slower; off by default)

Variable	Default	Controls
LLM_HEAD_SCORING_ENABLED	false	opt in to LLM <i>fit-scoring</i> the top ~250 (blended into the ranking)
LLM_SCORE_WEIGHT	0.15	blend weight for <code>llm_fit_score</code> in the head
COMPUTE_DEVICE	auto	transformer device: auto / cpu / cuda / mps
USE_ONNX	false	ONNX Runtime + int8 backend
NEO4J_ENABLED	false	use Neo4j (else in-memory graph)
GRAPH_HYBRID_ENABLED	true	hybrid graph+vector search
DATABASE_URL	sqlite	DB connection (Postgres in prod)
REDIS_URL	redis://...	cache + rate-limit + queue (falls back in-memory)
CELERY_TASK_ALWAYS_EAGER	true	run background tasks inline (no worker)
OTEL_ENABLED	false	OpenTelemetry tracing
JWT_SECRET	dev value	change in production
BOOTSTRAP_ADMIN_EMAIL/PASSWORD	admin@... / changeme-admin	first admin user

(Anything in `core/config.py` can be set this way — the env var is the upper-cased field name.)

Troubleshooting

Symptom	Cause & fix
IsADirectoryError: '/app/candidates.jsonl' (Docker)	No dataset file at the repo root → Docker made an empty dir. <code>docker compose down, rm -rf candidates.jsonl</code> , copy the real file, up again.
UI stuck on “Syncing...”	Cosmetic — the page doesn’t poll. Check the real status at <code>/developers/{id}/sync-status</code> or the server log (<code>sync.complete</code> vs <code>developer.sync_failed</code>).
GITHUB_TOKEN not configured in logs	The server process can’t see the token. Export it in the same shell that runs <code>uvicorn</code> (or the compose env) and restart. Verify with the one-liner in Section 8.
rank.py errors “feature matrix missing” torch install slow/large precompute “freezes” after <code>bm25.build</code>	Run <code>precompute.py</code> first (Section 1). Use the lean install (Section 0) + <code>precompute --no-st</code> . With a <code>GOOGLE_API_KEY</code> set, LLM label-grading used to run 75 slow Gemini calls here. It’s now off by default (heuristic labels, instant). Opt in with <code>PSEUDO_LABEL_LLM_ENABLED=true</code> if you want it.
Port already in use	Change <code>--port</code> (API) or Vite’s port in <code>frontend/vite.config.ts</code> .
beat crashes: Permission denied: 'celerybeat-schedule'	<code>/app</code> is root-owned but the container runs as <code>app</code> . Fixed in <code>compose</code> (writes the schedule to <code>/tmp</code>). Pull the latest <code>docker-compose.yml</code> .
neo4j exits (3) “again and again”	Usually a <i>cascade</i> : another container (e.g. <code>beat</code>) crashes, aborts <code>docker compose up</code> , and kills <code>neo4j</code> mid-startup; plus a stale data volume from earlier aborted runs. Fix: <code>docker compose down -v</code> then <code>docker compose up --build</code> . See the real reason with <code>docker compose logs neo4j</code> .
Reset local DB	<code>rm provenancerank.db*</code> (or make <code>clean</code>).

make targets at a glance

```

make install      # pip install -r requirements.txt
make precompute  # build offline artifacts
make rank         # produce submission.csv
make serve        # run the API (sqlite + in-memory)
make test         # run the test suite
make lint         # ruff + mypy

```

```
make migrate      # apply DB migrations
make up / down    # full Docker stack up / down
make logs         # tail API logs
make clean        # remove caches + local db
```

That's every section. Start at **Section 1** for the graded output; everything else layers on top without changing it.